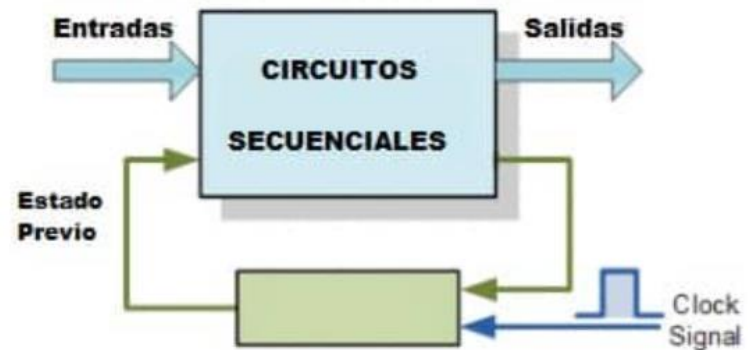
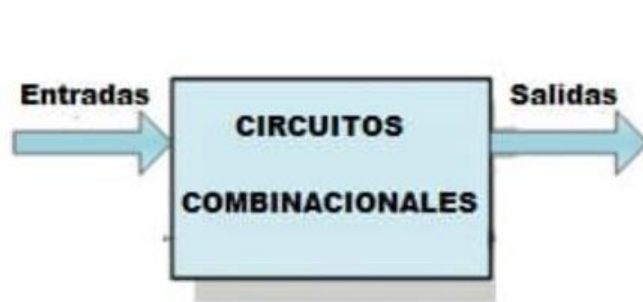


Introducción a VHDL

Matías Javier Oliva
Pablo García

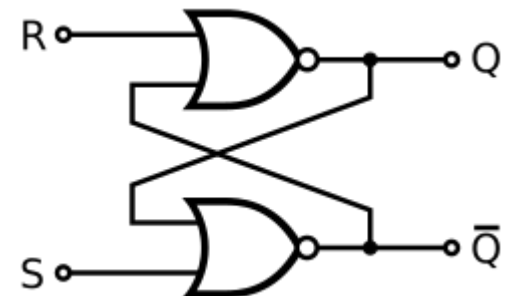
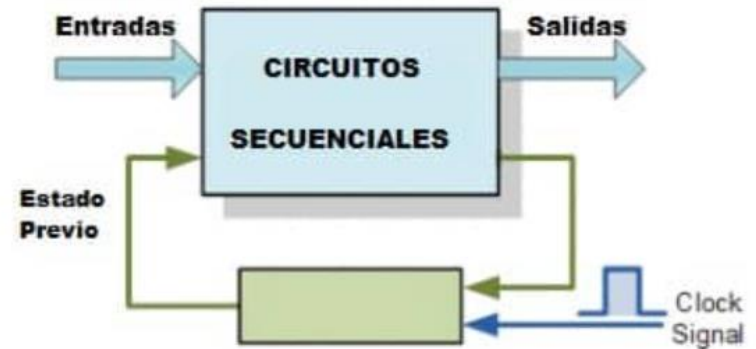
Lógica combinacional vs lógica secuencial

- Las funciones lógicas se pueden dividir en dos:
 - Por un lado están aquellas que solo dependen de los valores actuales de las señales: Lógica combinacional (compuertas and, or, xor, multiplexores...).
 - Por otro lado aquellas que utilizan memoria, es decir dependen de valores actuales y pasados de las señales: Lógica secuencial. Este tipo de lógica se basa en flip-flops (En general el FFD)



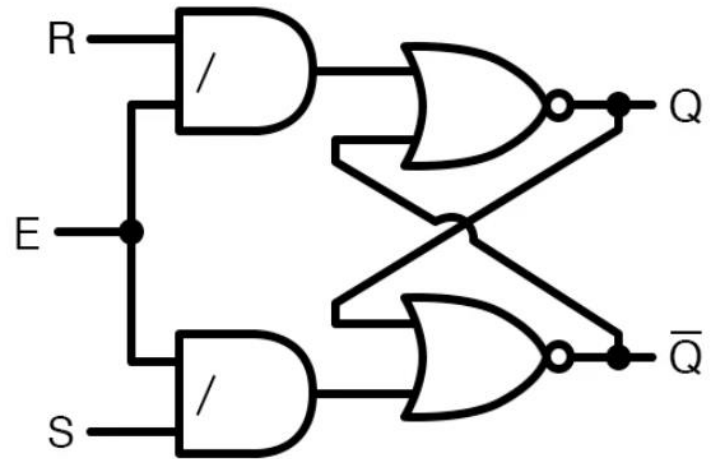
Lógica secuencial

- Si queremos diseñar un circuito que pueda reaccionar a estados previos de las entradas necesitamos un elemento que hasta ahora no hemos discutido: memoria.
- El tipo más simple de elemento de memoria es un biestable, llamado así porque puede estar en uno de dos estados estables: 0 o 1.
- Una vez que se encuentra en uno de estos estados, permanece allí hasta que algo lo cambie.
- El biestable más simple es el latch, que se puede construir a partir de compuertas NOR o NAND.



Biestables

- Un latch puede mantener un valor mientras se lo habilite con una señal de control (por ejemplo, un Enable).
- Sin embargo, puede cambiar su estado en cualquier momento en que estén habilitados, lo cual puede ser problemático.
- Para solucionar esto, se usa un flip-flop, que es un tipo de biestable que cambia su estado solo en un instante preciso del tiempo, normalmente en un flanco de reloj.
- Esto hace que su comportamiento sea más predecible y sincronizable con otros componentes.



Flip-flop tipo D

- El flip-flop tipo D (de Data o Delay) es un circuito que captura el valor de su entrada D en el instante del flanco activo del reloj (CLK), y mantiene ese valor hasta que ocurre un nuevo flanco.
- Podemos utilizar este elemento como **célula básica de memoria**.
- Uniendo varios flip-flop juntos podemos generar un **registro** de N bits.
- También se utiliza para sincronizar señales con el reloj.

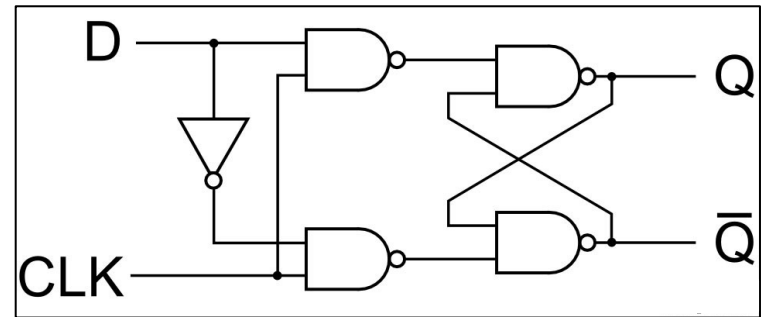


Tabla de verdad:

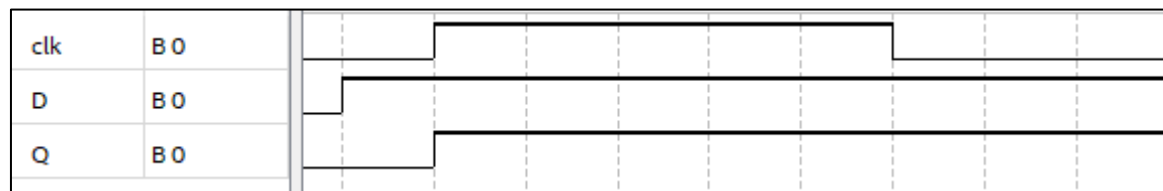
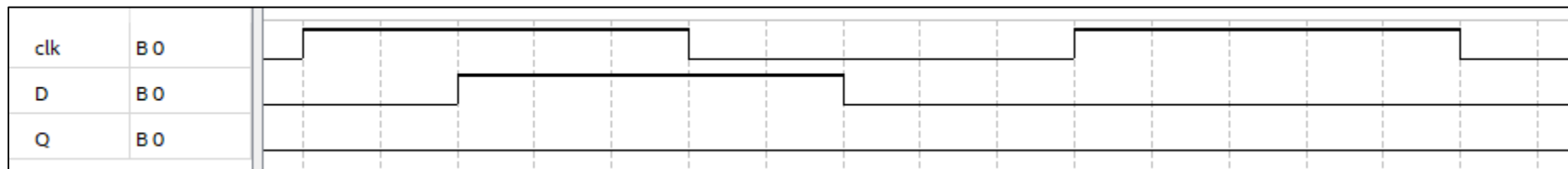
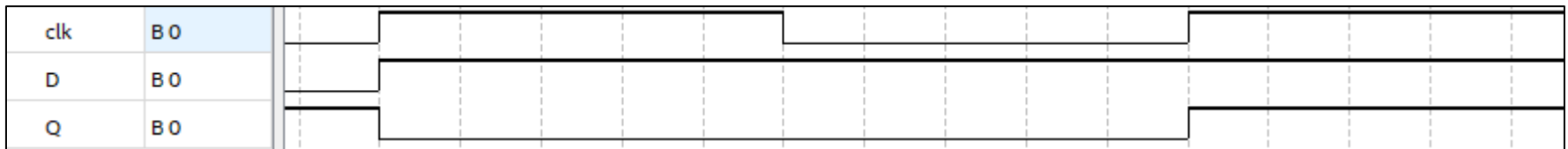
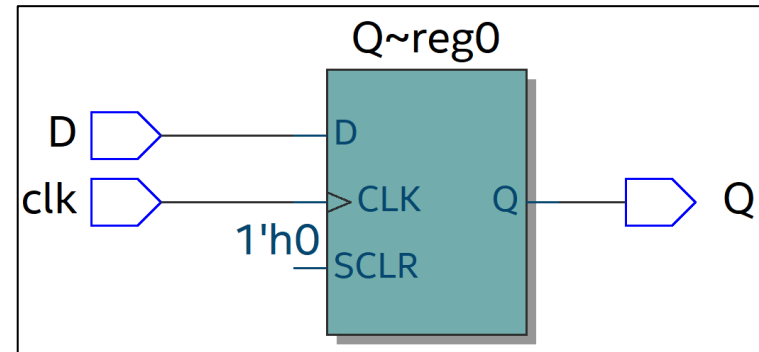
CLK	D	Q_{n+1}
↑	1	1
↑	0	0
-	-	Q_n

Flip-flop tipo D

```

architecture arch of clase3_a is
begin
  process(D,clk)
  begin
    if(rising_edge(clk)) then
      Q <= D;
    end if;
  end process;
end arch;

```



Flip-flop tipo D: Set y Reset

- Aparte de las señales D, clk y Q a los flip-flop se le suelen agregar entradas adicionales, que permiten imponer el estado del flip-flop sin utilizar la entrada D:
 - RESET: Impone un 0 en la salida del FF.
 - SET: Impone un 1 en la salida del FF.
- Estas entradas pueden estar subordinadas al reloj (sincrónicas) o no (asincrónicas).
- También pueden ser activas en alto o en bajo

Convención: Cuando son activas en bajo las señales se nombran con un `_n`



Ejemplo:
`reset_n`

Flip-flop tipo D: Set y Reset sincrónicos

```
library ieee;
use ieee.std_logic_1164.all;

entity Clase3_a is

    port(

        D: in std_logic;
        clk: in std_logic;
        rst: in std_logic;
        set: in std_logic;

        Q: out std_logic

    );
end entity;
```

```
architecture arch of Clase3_a is

begin

    process(D, clk)

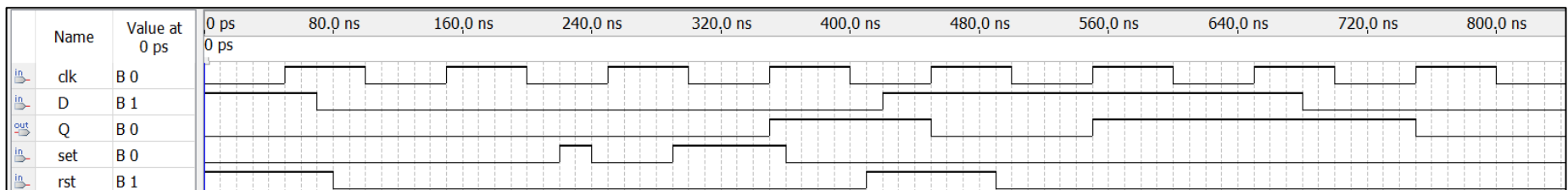
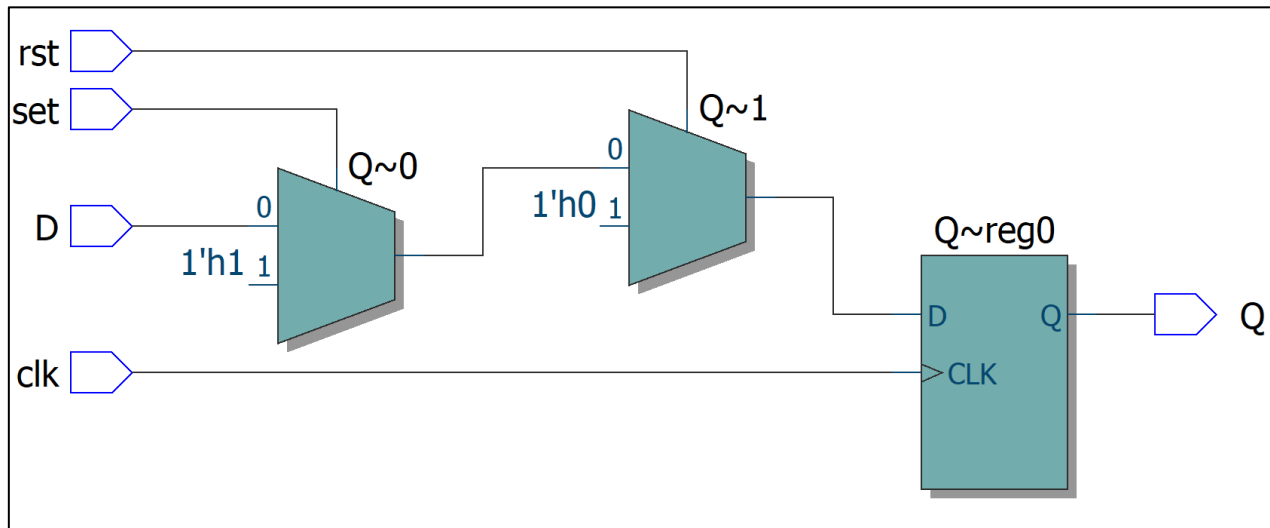
    begin

        if(rising_edge(clk)) then
            if(rst = '1') then
                Q <= '0';
            elsif(set = '1') then
                Q <= '1';
            else
                Q <= D;
            end if;
        end if;

    end process;

end arch;
```


Flip-flop tipo D: Set y Reset sincrónicos



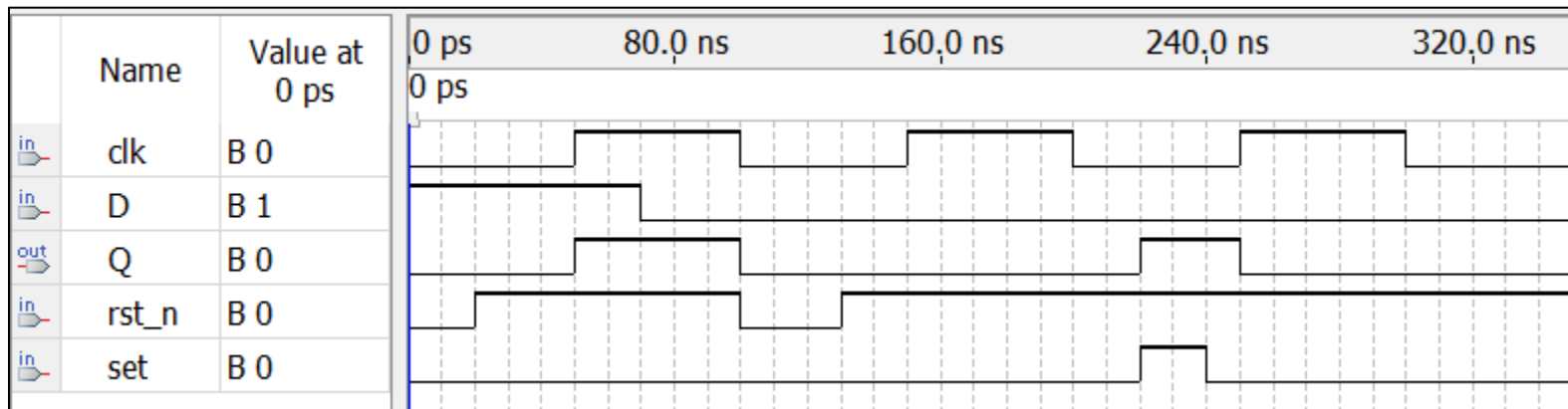
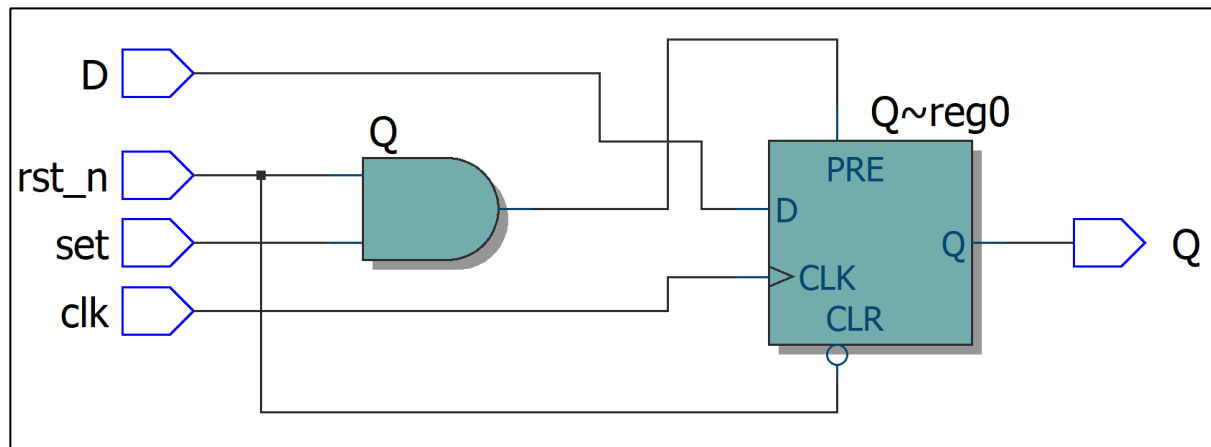
Flip-flop tipo D: Set y Reset asincrónicos

```
library ieee;
use ieee.std_logic_1164.all;

entity Clase3_a is
    port(
        D: in std_logic;
        clk: in std_logic;
        rst_n: in std_logic;
        set: in std_logic;
        Q: out std_logic
    );
end entity;
```

```
architecture arch of Clase3_a is
begin
    process(D,clk,rst_n,set)
    begin
        if(rst_n = '0') then
            Q <= '0';
        elsif(set = '1') then
            Q <= '1';
        elsif(rising_edge(clk)) then
            Q <= D;
        end if;
    end process;
end arch;
```

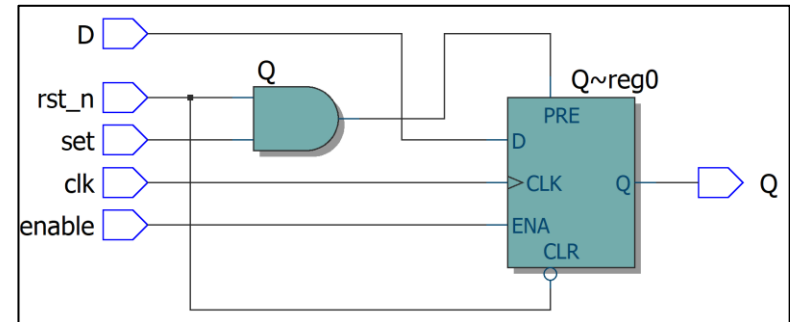
Flip-flop tipo D: Set y Reset asincrónicos



Flip- flop tipo D: Enable

- Una última señal que se le suele agregar a los FF es la señal de enable, que habilita o deshabilita su funcionamiento.

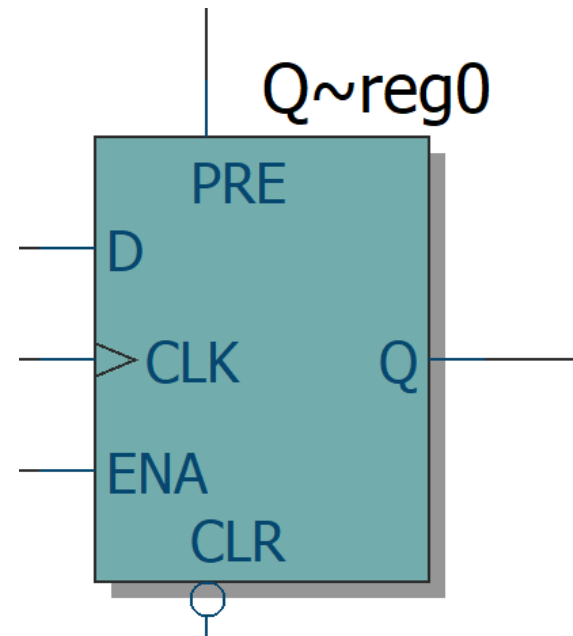
```
architecture arch of Clase3_a is
begin
  process(D,clk,rst_n,set)
  begin
    if(rst_n = '0') then
      Q <= '0';
    elsif(set = '1') then
      Q <= '1';
    elsif(enable = '1' and rising_edge(clk)) then
      Q <= D;
    end if;
  end process;
end arch;
```



- La prioridad de las señales depende del diseñador, pero en general se toma la indicada en el ejemplo anterior: reset – set – enable – D.

Flip-flop D completo

```
entity ff_d is  
    port(  
        D: in std_logic;  
        clk: in std_logic;  
        rst_n: in std_logic;  
        set: in std_logic;  
        enable : in std_logic;  
  
        Q: out std_logic  
    );  
end entity;
```



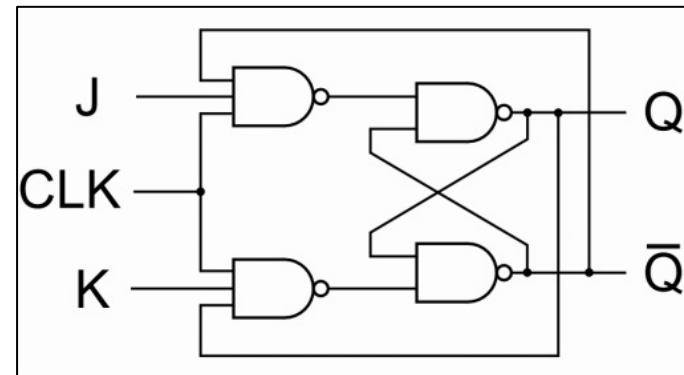
¡Conviene tener la entidad a mano para usarla
como componente en distintos diseños!

Otros flip-flop: Tipo JK

- Si bien el flip-flop tipo D es el más utilizado (y el único que sintetiza Quartus directamente), existen otros tipos de flip-flop.
- En este curso vamos a ver brevemente el JK, el SR y el T.

Tabla de verdad:

CLK	J	K	Q_{n+1}
↑	0	0	Q_n
↑	1	0	1
↑	0	1	0
↑	1	1	$\overline{Q_n}$
↓	-	-	Q_n



El comportamiento de Toogle resulta conveniente en algunas aplicaciones!.

Otros flip-flop: Tipo JK

```

architecture arch of Clase3_b is
    signal q_reg: std_logic := '0'; -- Registro interno
    signal jk : std_logic_vector (1 downto 0);
begin

    jk <= J & K;
    process(clk, rst_n, set)
    begin
        if rst_n = '0' then
            q_reg <= '0';
        elsif set = '1' then
            q_reg <= '1';
        elsif rising_edge(clk) then
            if enable = '1' then
                case (jk) is
                    when "00" => q_reg <= q_reg; -- sin cambio
                    when "01" => q_reg <= '0'; -- reset
                    when "10" => q_reg <= '1'; -- set
                    when "11" => q_reg <= not q_reg; -- toggle
                    when others => null;
                end case;
            end if;
        end if;
    end process;

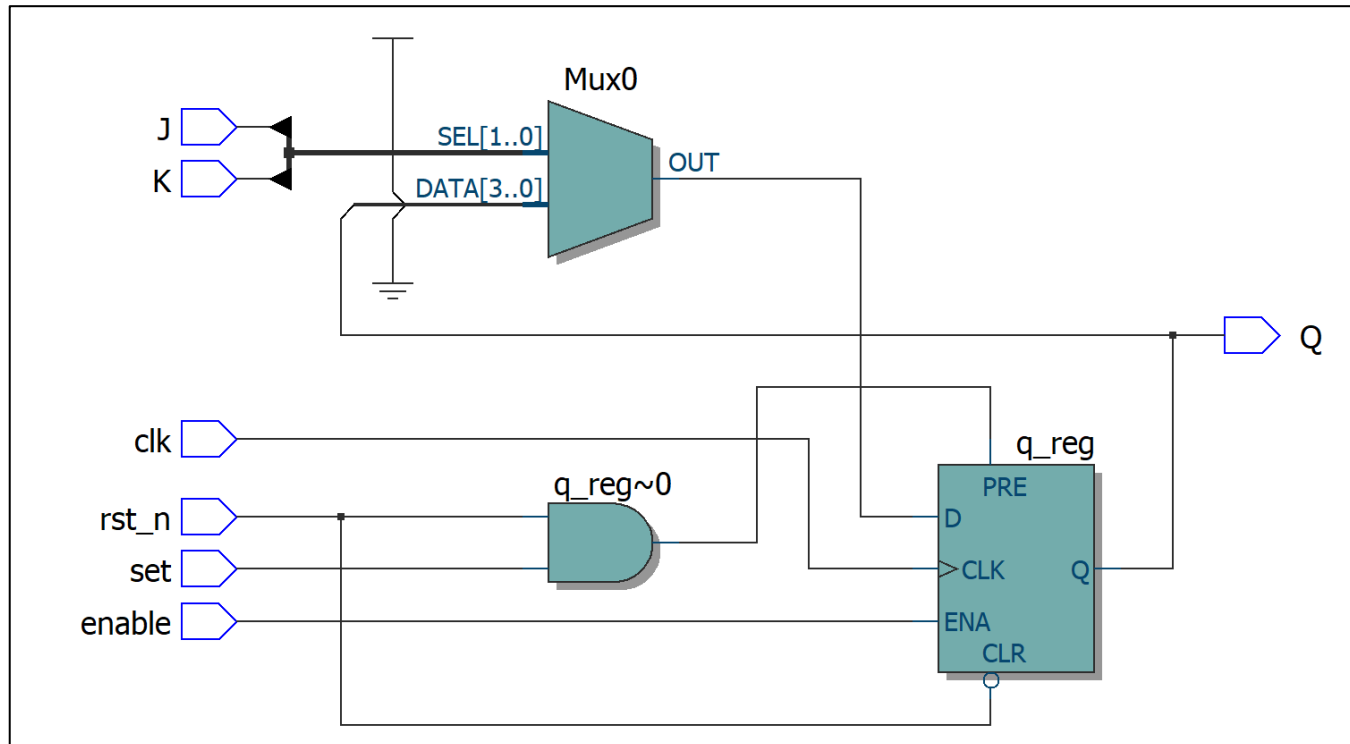
    Q <= q_reg;
end arch;

```

- Es necesaria una señal auxiliar q_reg para poder leer la salida desde el diseño.
- Sin eso no se podría hacer $Q \leq \text{not}(Q)$.
- Otra señal auxiliar jk se uso para simplificar la sintaxis

Otros flip-flop: Tipo JK

- Quartus no implementa el flip-flop JK directamente, sino que le agrega lógica a un flip-flop tipo D.

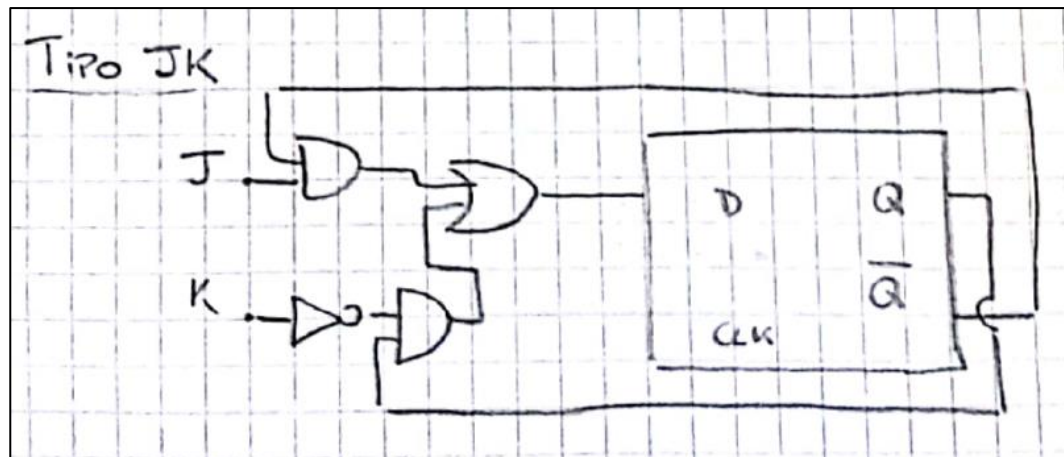


Flip-flop JK a partir de uno D

J	0 0	0 0	1 1	1 1
K	0 0	1 1	0 0	1 1
Q_p	0 1	0 1	0 1	0 1
$Q_{p+1} \equiv D$	0 1	0 0	1 1	1 0

Q_p	00	01	11	10
J		1		
K	1	1		1

$\Rightarrow D = \bar{K} Q_p + J \bar{Q}_p$



Otros flip-flop: Tipo T

- También llamado tipo Toogle, si su entrada T esta en '1' su salida alterna entre estados.
- Se puede construir fácilmente a partir de uno JK.

Tabla de verdad:

CLK	T	Q_{n+1}
↑	0	Q_n
↑	1	$\overline{Q_n}$
↓	-	Q_n

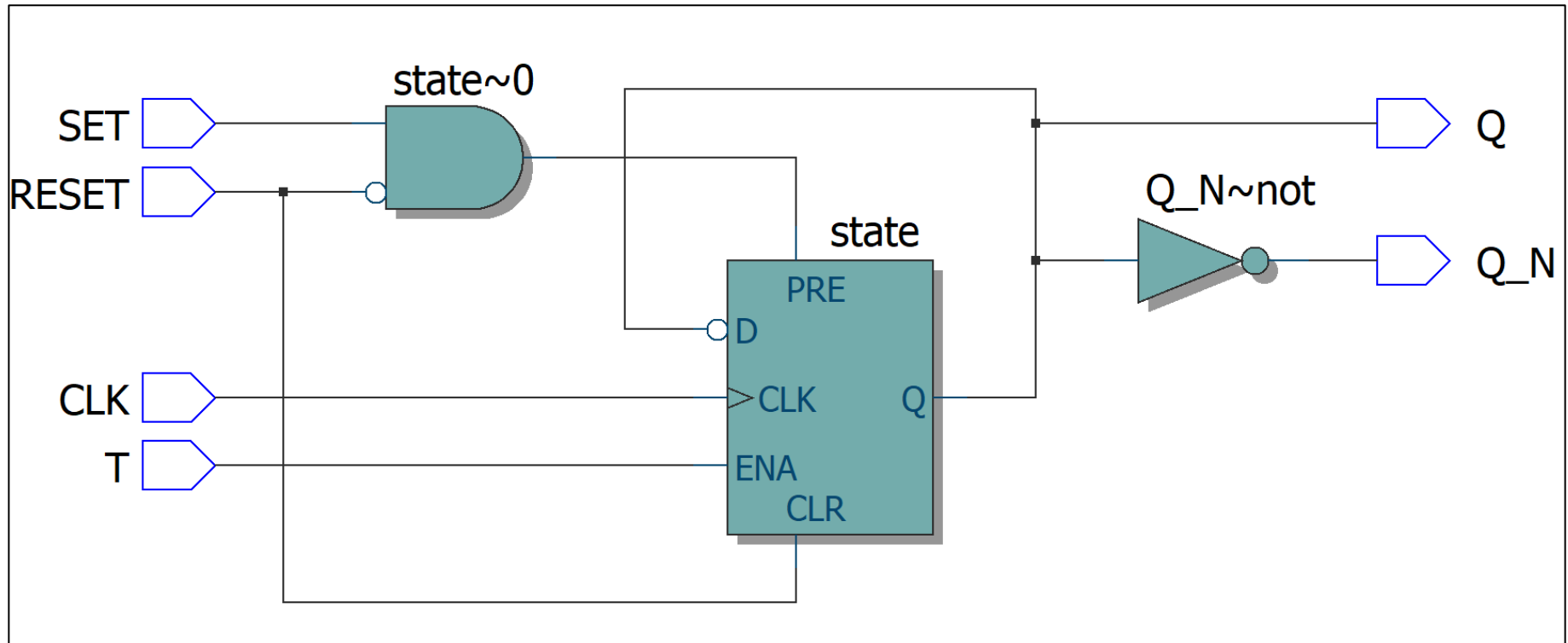
```
architecture Behavioral of Clase3_c is
    signal state : STD_LOGIC := '0';
begin
    process(CLK, RESET, SET)
    begin
        if RESET = '1' then
            state <= '0';

        elsif SET = '1' then
            state <= '1';

        elsif rising_edge(CLK) then
            if T = '1' then
                state <= not state;
            end if;
        end if;
    end process;

    Q <= state;
    Q_n <= not state;
end Behavioral;
```

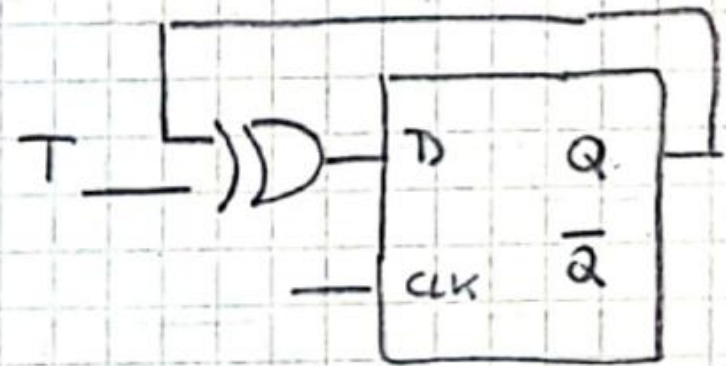
Otros flip-flop: Tipo T



Síntesis de un flip-flop T a partir de un D

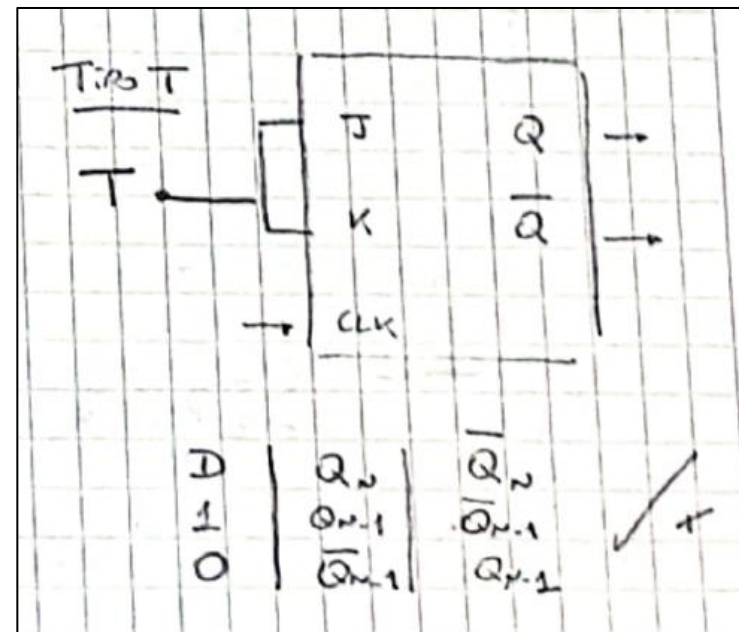
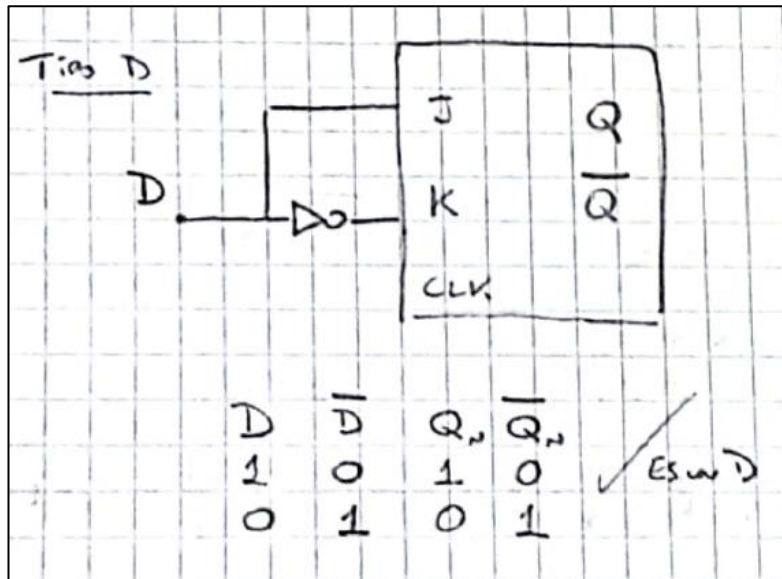
Tipo T $\Rightarrow D = \overline{T} \cdot Q_p + T \cdot \overline{Q_p} = T \oplus Q_p$

T	0	0	1	1
Q _p	0	1	0	1
	Q _p	Q _p	$\overline{Q_p}$	$\overline{Q_p}$
Q _{p+1} = D	0	1	1	0



- Notar que Quartus lo implementa de otra manera, haciendo uso de la entrada enable del flip-flop tipo D.

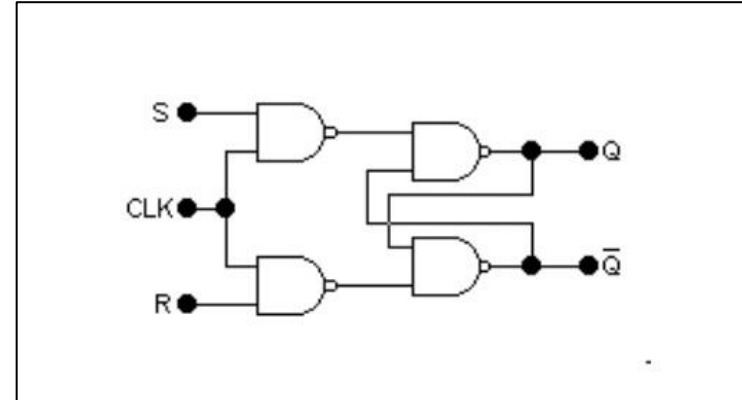
Otras “conversiones” de Flip-flops



Otros flip-flop: Tipo SR

Tabla de verdad:

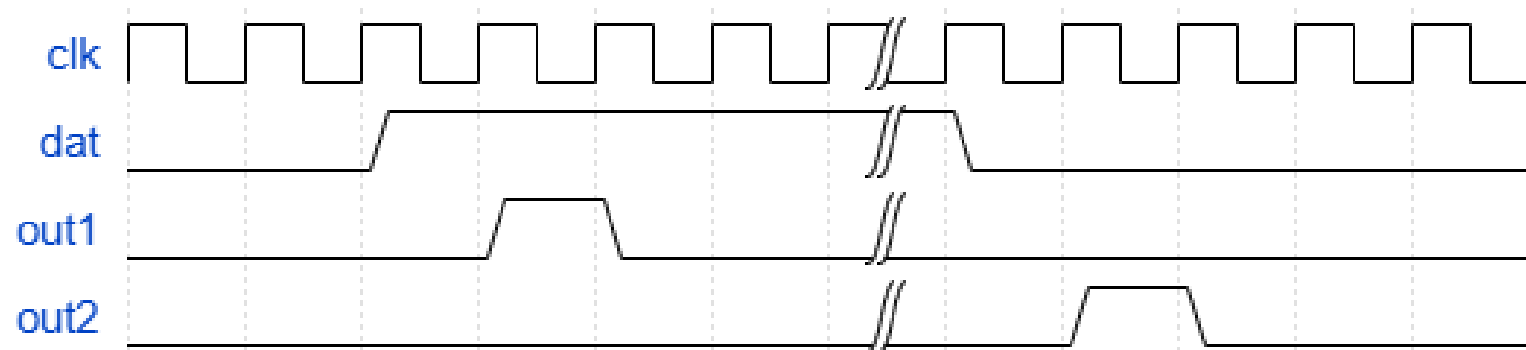
CLK	S	R	Q_{n+1}
↑	0	0	Q_n
↑	0	1	0
↑	1	0	1
↑	1	1	Prohibido
↓	-	-	Q_n



- SR por Set Reset.
- El estado prohibido hace que este flip-flop sea problemático.

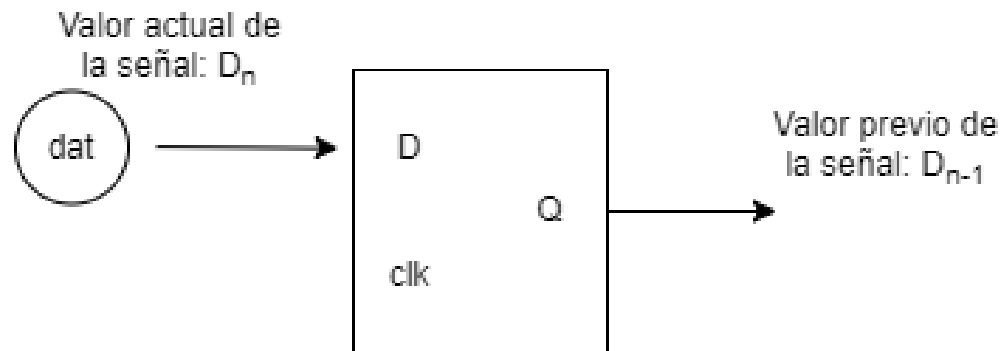
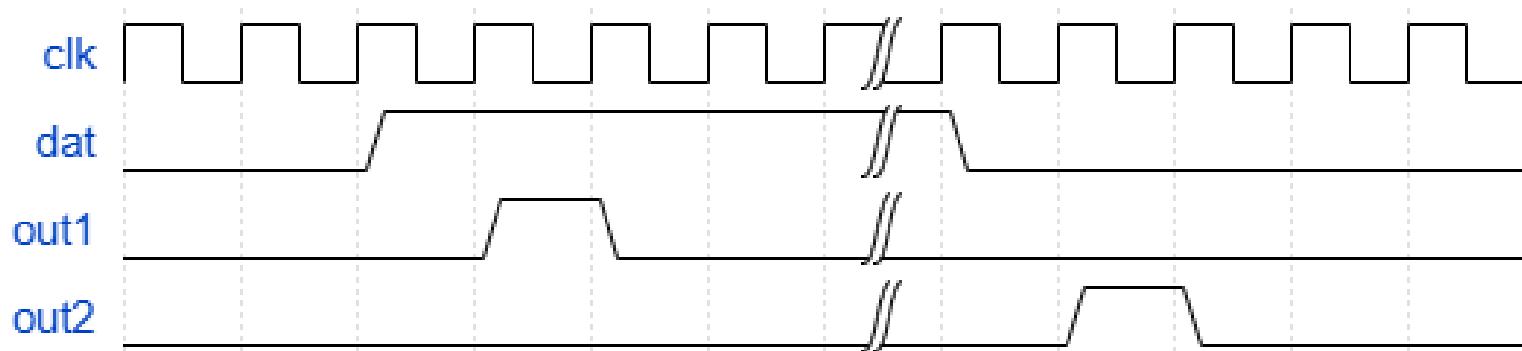
Detector de flancos

- Un ejemplo que sirve para mostrar (y entender) la utilidad de un flip-flop tipo D es la detección de un flanco en alguna señal.
- En primer lugar notemos que esto sería imposible con lógica combinacional, porque solo tendríamos acceso al **valor actual** de la señal y no podemos saber como era antes.



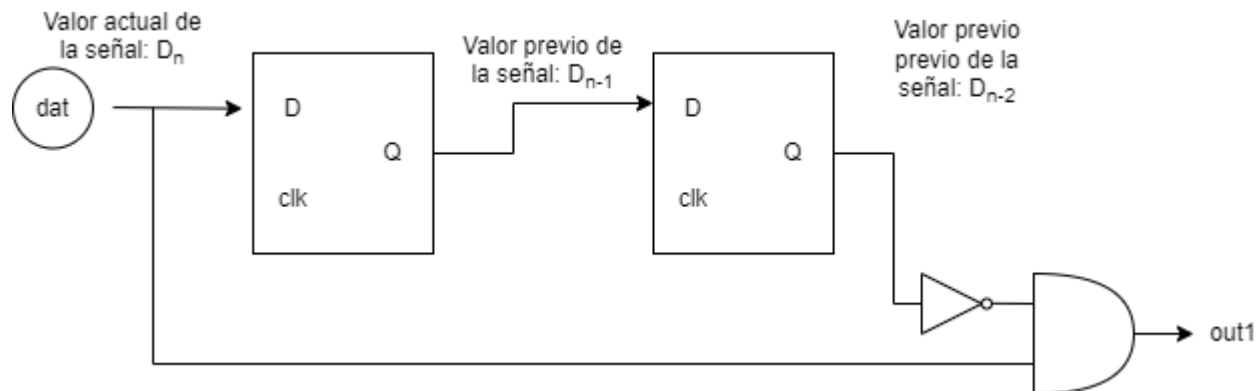
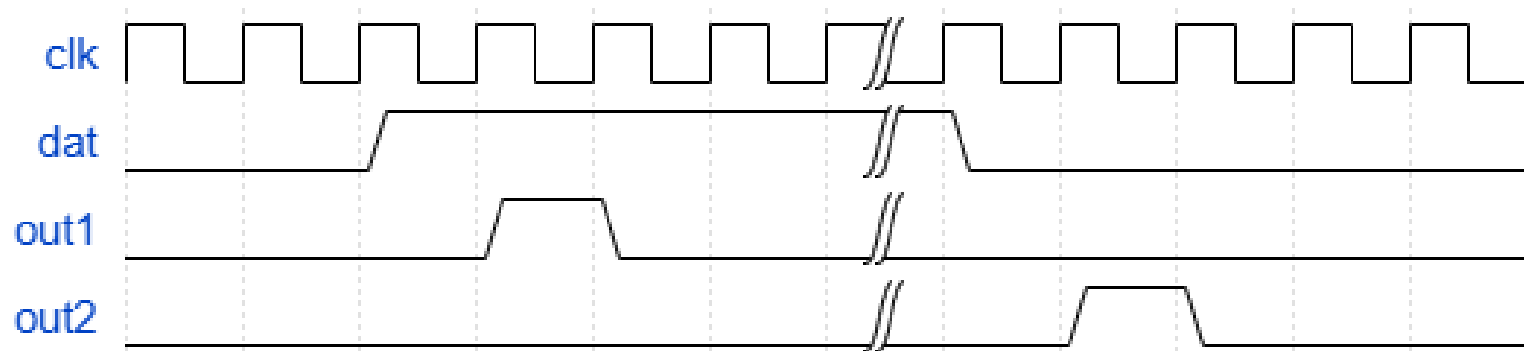
- out1: Detector de flancos de subida.
- out2: Detector de flancos de bajada

Detector de flancos



- Una vez que se entiende esto solo resta generar la señal deseada haciendo operaciones combinatoriales sobre D_n y D_{n-1}

Detector de flancos



- Agregamos un flip-flop extra para garantizar la sincronización

Detector de flancos

```

architecture arch of Clase3_d is
  signal dat_1, dat_2 : std_logic := '0';

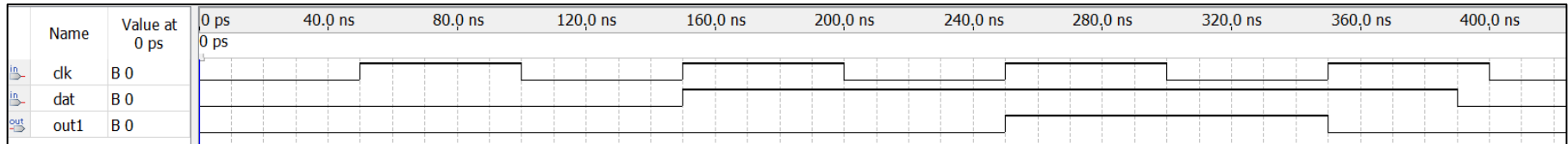
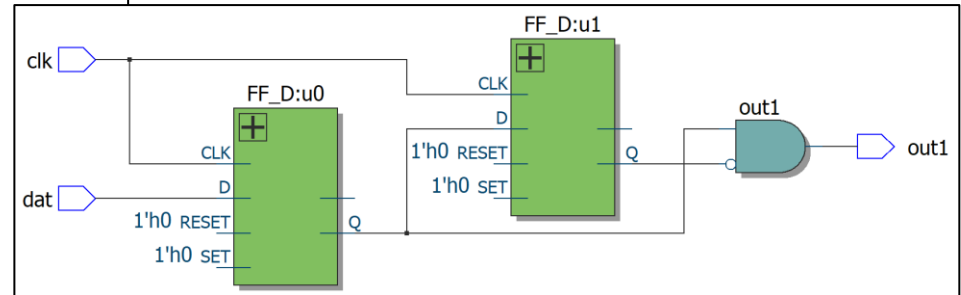
  component FF_D is
    Port (
      D, CLK, SET, RESET : in STD_LOGIC;
      Q, Q_N              : out STD_LOGIC
    );
  end component;

begin
  u0: FF_D port map (dat, clk, '0', '0', dat_1, open);
  u1: FF_D port map (dat_1, clk, '0', '0', dat_2, open);

  out1 <= not(dat_2) and dat_1;

end arch;

```



Lógica secuencial: Otras sintaxis

```

library ieee;
use ieee.std_logic_1164.all;

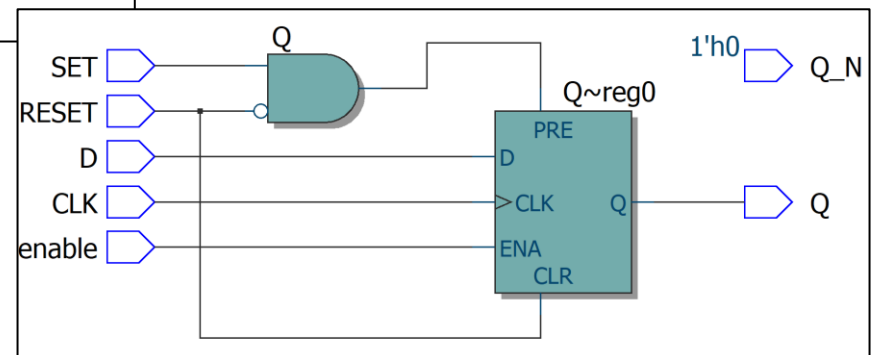
entity Clase3_e is
    Port (
        D, CLK, SET, RESET, enable : in STD_LOGIC;
        Q, Q_N                     : out STD_LOGIC
    );
end Clase3_e;

architecture Behavioral of Clase3_e is
begin
    q <= '0' when RESET = '1' else
        '1' when SET = '1' else
        d when clk'event and clk = '1' and enable = '1';

end Behavioral;

```

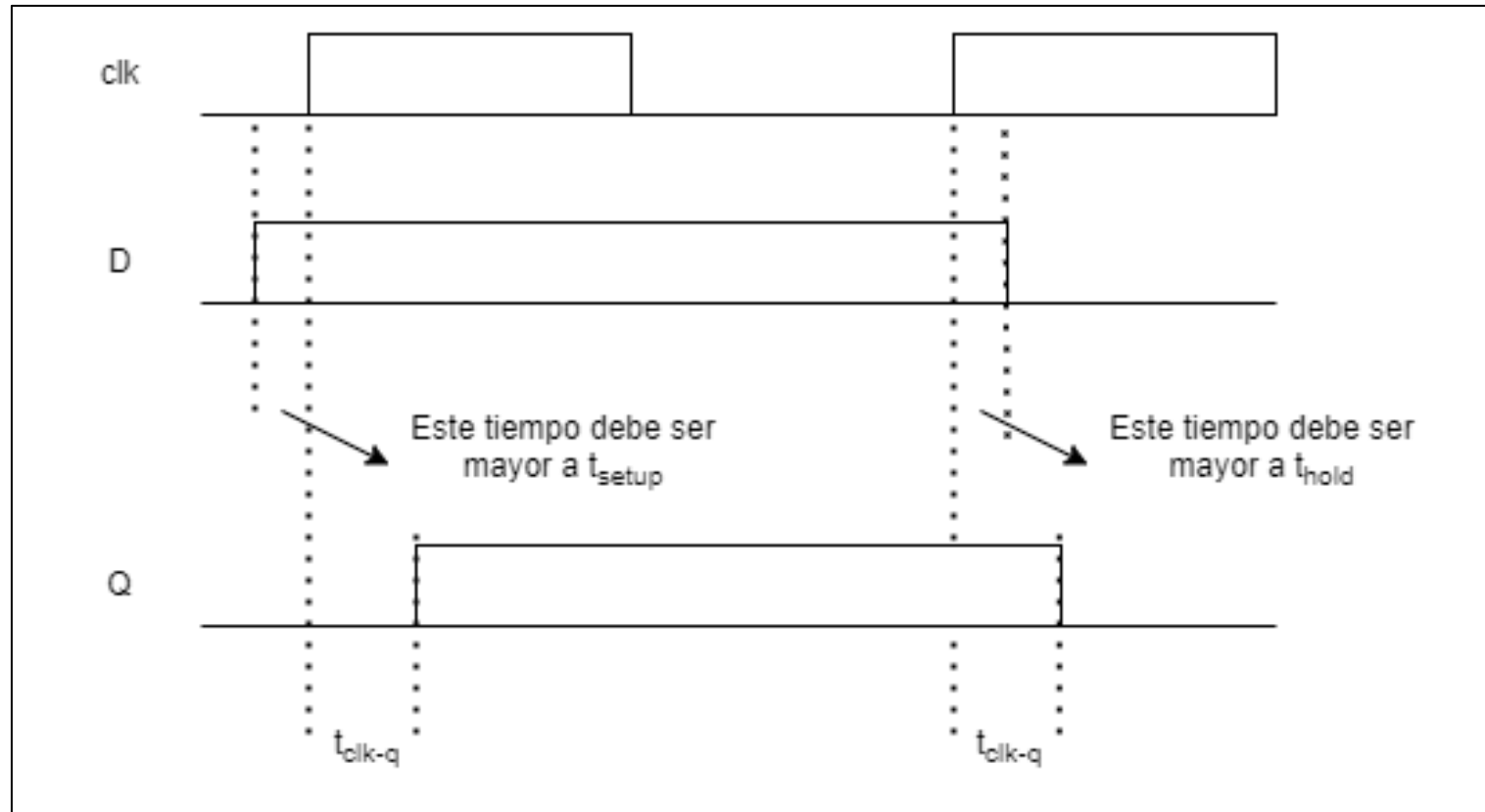
- También es posible inferir lógica secuencial con una sintaxis concurrente como esta.
- Resulta menos común.



Flip-flop reales: tiempos de retardo

- Cuando trabajamos con lógica secuencial un aspecto que toma mucha relevancia es que tan rápido pueden funcionar nuestros diseños.
- Para calcular esto en los flip-flop (registros) se definen algunos retardos importantes:
 - $t_{\text{clk-q}}$: t que tarda en propagarse la señal desde D hasta Q medida desde que llega el flanco de clk.
 - t_{setup} : t que debe estar estable la entrada D antes del flanco.
 - t_{hold} : t que debe estar estable la entrada D después del flanco.

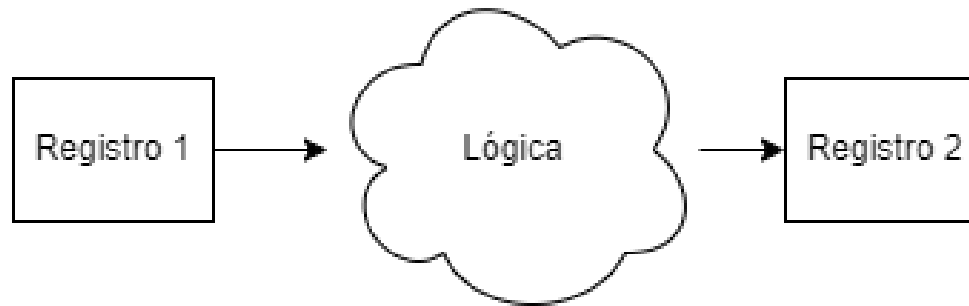
Flip-flop reales: tiempos de retardo



En general t_{hold} resulta más chico que $t_{\text{clk-q}}$ por lo que suele ignorar.

Herramientas de análisis temporal

- Al implementar un diseño secuencial estamos generalmente en una situación como esta:



- La lógica entre registros puede hacer que se violen alguno de los tiempos críticos de los flip-flops.
- Otra forma de verlo es que hay una frecuencia de reloj máxima a la que puede utilizarse un diseño antes de que se violen estos tiempos.
- Quartus (y otras herramientas) proveen herramientas para analizar el peor caso (camino crítico).

Synopsys Design Constraints

- Para analizar temporalmente un diseño primero se debe crear un archivo de Synopsys Design Constraints (.sdc).
- Con este se definen que pines son relojes, y a que velocidad operan, entre otras cosas. Por ejemplo:

```
# archivo: Clase3_d.sdc  
  
# Definición del reloj de entrada (por ejemplo, 10 ns = 100 MHz)  
create_clock -name clk -period 10.0 [get_ports clk]
```

- Le dice a Quartus que nuestro diseño tiene un reloj que va a llamar clk que tiene que asociar a un puerto clk.
- El archivo de .sdc puede servir para especificar muchas otras cosas, por ejemplo que caminos ignorar, cual es el retardo de determinados puertos i/o...

Herramientas de análisis temporal

- Una vez que el diseño tiene un .sdc al compilarlo se genera un reporte del Timing TimeQuestAnalyzer, que analiza si nuestro diseño pudo cumplir con las condiciones temporales dadas.
- También especifica que tan rápido podría llevarse el reloj sin violar las condiciones temporales.
- Todo esto se analiza bajo distintas condiciones de tensión y temperatura.

Slow 1200mV 85C Model Fmax Summary

	Fmax	Restricted Fmax	Clock Name	Note
1	1101.32 MHz	250.0 MHz	clk	limit due to minimum period restriction (max I/O toggle rate)

Slow 1200mV 0C Model Fmax Summary

	Fmax	Restricted Fmax	Clock Name	Note
1	1228.5 MHz	250.0 MHz	clk	limit due to minimum period restriction (max I/O toggle rate)

- La restricted Fmax esta influenciada por cosas externas a nuestro diseño (en este caso la máxima reestriccion de puertos i/o)